Lab 2 - Encapsulation, Methods, Arrays, and Static Variables

This lab introduces essential object-oriented programming concepts in Java through three progressive tasks. Students will learn encapsulation by implementing private variables with getter/setter methods, work with arrays and static variables for data management, and practice method design with validation. The exercises build from enhancing the Lab 1 name management system to creating grade calculators and bank account simulators, emphasizing proper class design, data protection, and modular programming principles.

Contents

Memorize

Submission Deadline

Deadline to upload the solutions for all tasks is Saturday, 11:59 pm before the lab date.

General Information

The following tasks are to be worked on in fixed teams of two. Each team member must be able to explain all solutions. Please submit only one solution for each team of two. The submission must be a PDF file in our Moodle room with the name and matriculation number. Solutions must be in digital format with intermediate steps and detailed explanations (no handwritten scans). You can use any tool or drawing program of your choice to create the diagrams. If you have questions or need support, use the forum in our Moodle room and help each other.

1. Task 1: Enhanced Name Management System

Extend your Java program from Lab 1 to use proper object-oriented programming principles. This task builds on the Person class you created and adds encapsulation, class variables, and arrays.

Modify your name management program as follows:

- Make all instance variables in the Person class private
- Create getter and setter methods to access these variables
- Add validation in the setter for month (must be between 1-12)
- Replace the three separate person variables with an array of Person objects
- Add a static class variable to count the total number of Person objects
- Extend the menu to include option 5: display the total person count

Detailed requirements:

- The inputs in the individual input fields themselves do not need to be checked again for correctness, i.e., letters, numbers, special characters, meaning the inputs in the individual input fields must be meaningful and do not need to be checked by your program. However, all instance variables must now be protected from direct external access, i.e., they get the private attribute and access to them is done via so-called getter and setter methods. Only the month input needs to be exemplarily checked for plausibility.
- Introduce a class variable that counts the number of persons and outputs it on request, i.e., with the input of the number 5 in the selection menu, the number of persons is output. (This is still constant in this task, but could become significant in a later extension.)
- Furthermore, the individual persons are now to be stored in an array and no longer in separate variables, to enable an extension to a realistic number of persons.

1.1. Preparation

Again, first clarify the task by drawing the Person class according to UML notation and then creating a structure chart or flowchart that solves this task. Then define all necessary classes, methods, and variables in Java.

1.2. Assistance

The following code snippets demonstrate key concepts needed for this task:

Private variables and getter/setter methods:

```java
public class Person {
    private String firstName;
    private String lastName;
    private int day, month, year;
    private static int personCount = 0;

    // Constructor
    public Person(String firstName, String lastName) {
        this.firstName = firstName;
        this.lastName = lastName;
        personCount++; // Increment when new person is created
    }

    // Getter methods
    public String getFirstName() {
        return firstName;
    }

    // Setter with validation
    public void setMonth(int month) {
        if (month >= 1 && month <= 12) {
            this.month = month;
        } else {
            System.out.println("Invalid month! Must be between 1-12.");
        }
    }

    // Static method to get person count
    public static int getPersonCount() {
        return personCount;
    }
}
```

Using arrays instead of separate variables:

```java
Person[] persons = new Person[3];
persons[0] = new Person("John", "Doe");
persons[1] = new Person("Jane", "Smith");
persons[2] = new Person("Bob", "Johnson");
```

1.3. Solution

Person.java:

```java
public class Person {
  private String firstName;
  private String lastName;
  private int day;
  private int month;
  private int year;
  private static int personCount = 0;

  // Constructor
  public Person(String firstName, String lastName, int day, int month, int year) {
    this.firstName = firstName;
    this.lastName = lastName;
    this.day = day;
    this.month = month;
    this.year = year;
    personCount++;
  }

  // Getter methods
  public String getFirstName() {
    return firstName;
  }

  public String getLastName() {
    return lastName;
  }

  public int getDay() {
    return day;
  }

  public int getMonth() {
    return month;
  }

  public int getYear() {
    return year;
  }

  // Setter methods
  public void setFirstName(String firstName) {
    this.firstName = firstName;
```

```
43    }
44
45    public void setLastName(String lastName) {
46      this.lastName = lastName;
47    }
48
49    public void setDay(int day) {
50      this.day = day;
51    }
52
53    // Setter with validation for month
54    public void setMonth(int month) {
55      if (month >= 1 && month <= 12) {
56        this.month = month;
57      } else {
58        System.out.println("Invalid month! Must be between 1-12.");
59      }
60    }
61
62    public void setYear(int year) {
63      this.year = year;
64    }
65
66    // Static method to get person count
67    public static int getPersonCount() {
68      return personCount;
69    }
70
71    // Display person information
72    public void displayInfo() {
73      System.out.println("Name: " + firstName + " " + lastName);
74      System.out.println("Name change date: " + day + "." + month + "." + year);
75    }
76
77    // Check if three years have passed
78    public boolean canChangeName(int newDay, int newMonth, int newYear) {
79      int yearsDiff = newYear - this.year;
80
81      if (yearsDiff < 3) {
82        return false;
83      }
84
85      if (yearsDiff == 3) {
```

```
86        if (newMonth < this.month) {
87          return false;
88        }
89        if (newMonth == this.month && newDay < this.day) {
90          return false;
91        }
92      }
93
94      return true;
95    }
96  }
```

EnhancedNameManagement.java:

```java
import java.util.Scanner;

public class EnhancedNameManagement {
  public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);

    // Hamburg greeting
    System.out.println("Moin!");

    // Create array of Person objects
    Person[] persons = new Person[3];
    persons[0] = new Person("Max", "Mustermann", 1, 1, 2020);
    persons[1] = new Person("Anna", "Schmidt", 15, 6, 2019);
    persons[2] = new Person("Peter", "Mueller", 20, 12, 2021);

    boolean running = true;

    while (running) {
      System.out.println("\n=== Enhanced Name Management System ===");
      System.out.println("Which person wants to change their name?");
      System.out.println("Enter 1, 2, or 3 for the person");
      System.out.println("Enter 4 to display all persons");
      System.out.println("Enter 5 to display total person count");
      System.out.println("Enter 0 to exit");
      System.out.print("Your choice: ");

      int choice = scanner.nextInt();
      scanner.nextLine(); // Consume newline

      if (choice == 0) {
```

```
31          System.out.println("Program terminated.");
32          running = false;
33        } else if (choice == 4) {
34          // Display all persons
35          System.out.println("\n=== All Persons ===");
36          for (int i = 0; i < persons.length; i++) {
37            System.out.println("\nPerson " + (i + 1) + ":");
38            persons[i].displayInfo();
39          }
40        } else if (choice == 5) {
41          // Display person count
42          System.out.println("\nTotal number of persons: " +
    Person.getPersonCount());
43        } else if (choice >= 1 && choice <= 3) {
44          // Select the person to change
45          Person selectedPerson = persons[choice - 1];
46
47          // Get new name information
48          System.out.print("Enter new first name: ");
49          String newFirst = scanner.nextLine();
50          selectedPerson.setFirstName(newFirst);
51
52          System.out.print("Enter new last name: ");
53          String newLast = scanner.nextLine();
54          selectedPerson.setLastName(newLast);
55
56          System.out.print("Enter day of name change: ");
57          int newDay = scanner.nextInt();
58          selectedPerson.setDay(newDay);
59
60          System.out.print("Enter month of name change (1-12): ");
61          int newMonth = scanner.nextInt();
62          selectedPerson.setMonth(newMonth); // Validates month automatically
63
64          System.out.print("Enter year of name change: ");
65          int newYear = scanner.nextInt();
66          scanner.nextLine(); // Consume newline
67
68          // Check if three years have passed
69          if (selectedPerson.canChangeName(newDay, newMonth, newYear)) {
70            selectedPerson.setYear(newYear);
71            System.out.println("\nName change successful!");
72            selectedPerson.displayInfo();
```

```
73          } else {
74            System.out.println("\nError: Three years have not passed since last name change!");
75          }
76        } else {
77          System.out.println("Invalid choice!");
78        }
79      }
80
81      scanner.close();
82    }
83  }
```

2. Task 2: Student Grade Calculator (Continuation of Lab 1, Task 4)

2.1. Preparation

Before implementing the solution, you must:

1. Design a class structure for the grade calculator system
2. Create a UML class diagram showing:
   - Class name (e.g., GradeCalculator)
   - All methods with their parameters and return types
   - Any necessary attributes/fields
3. Transfer your implementation into this class structure
4. Only after completing the UML diagram should you begin coding

This preparation step ensures you think about the object-oriented design before writing code.

2.2. Task

This task continues the grade management system from Lab 1, Task 4. Create a Java program that manages student grades using arrays and methods. This task will help you practice working with arrays, methods, and calculations.

Write a program that:

- Stores grades for 5 students in an array
- Calculates the average grade
- Finds the highest and lowest grades
- Counts how many students passed (grade >= 60)
- Displays all results in a clear format

Your program should:

1. Ask the user to enter 5 student grades (0-100)
2. Store these grades in an array
3. Use separate methods for each calculation:
   - `int calculateAverage(int[] grades)` - returns the average
   - `int findHighest(int[] grades)` - returns the highest grade
   - `int findLowest(int[] grades)` - returns the lowest grade
   - `int countPassing(int[] grades)` - returns number of passing grades
   - `int[] deduplicatedGrades(int[] grades)` - returns a deduplicated array of occurred grades
4. Display all results with appropriate labels

2.3. Requirements

- Use an array to store the grades
- Create separate methods for each calculation
- Use loops to process the array
- Display results with clear formatting

2.4. Assistance

Array declaration and initialization:

```java
int[] grades = new int[5];
```

```
2
3  // Reading grades into array
4  Scanner scanner = new Scanner(System.in);
5  for (int i = 0; i < grades.length; i++) {
6      System.out.print("Enter grade for student " + (i + 1) + ": ");
7      grades[i] = scanner.nextInt();
8  }
```

Method example for calculating average:

```
1  public static double calculateAverage(int[] grades) {                Java
2      int sum = 0;
3      for (int grade : grades) {
4          sum += grade;
5      }
6      return (double) sum / grades.length;
7  }
```

2.5. Solution

```
1    import java.util.Scanner;                                           Java
2
3    public class GradeCalculator {
4      public static void main(String[] args) {
5        Scanner scanner = new Scanner(System.in);
6
7        // Create array for 5 students
8        int[] grades = new int[5];
9
10       // Input grades
11       System.out.println("=== Student Grade Calculator ===");
12       for (int i = 0; i < grades.length; i++) {
13         System.out.print("Enter grade for student " + (i + 1) + ": ");
14         grades[i] = scanner.nextInt();
15       }
16
17       // Calculate statistics using methods
18       double average = calculateAverage(grades);
19       int highest = findHighest(grades);
20       int lowest = findLowest(grades);
21       int passingCount = countPassing(grades);
22       int[] uniqueGrades = deduplicatedGrades(grades);
23
24       // Display results
25       System.out.println("\n=== Grade Statistics ===");
```

```
26      System.out.println("Average: " + average);
27      System.out.println("Highest: " + highest);
28      System.out.println("Lowest: " + lowest);
29      System.out.println("Passing (>= 60): " + passingCount);
30      System.out.println("Failing (< 60): " + (grades.length - passingCount));
31
32      System.out.print("Unique grades occurred: ");
33      for (int i = 0; i < uniqueGrades.length; i++) {
34        System.out.print(uniqueGrades[i]);
35        if (i < uniqueGrades.length - 1) {
36          System.out.print(", ");
37        }
38      }
39      System.out.println();
40
41      scanner.close();
42    }
43
44    // Calculate average grade
45    public static double calculateAverage(int[] grades) {
46      int sum = 0;
47      for (int grade : grades) {
48        sum += grade;
49      }
50      return (double) sum / grades.length;
51    }
52
53    // Find highest grade
54    public static int findHighest(int[] grades) {
55      int highest = grades[0];
56      for (int grade : grades) {
57        if (grade > highest) {
58          highest = grade;
59        }
60      }
61      return highest;
62    }
63
64    // Find lowest grade
65    public static int findLowest(int[] grades) {
66      int lowest = grades[0];
67      for (int grade : grades) {
68        if (grade < lowest) {
```

```
69            lowest = grade;
70          }
71        }
72        return lowest;
73      }
74
75      // Count passing grades (>= 60)
76      public static int countPassing(int[] grades) {
77        int count = 0;
78        for (int grade : grades) {
79          if (grade >= 60) {
80            count++;
81          }
82        }
83        return count;
84      }
85
86      // Return deduplicated array of grades
87      public static int[] deduplicatedGrades(int[] grades) {
88        // First, count unique grades
89        int uniqueCount = 0;
90        int[] temp = new int[grades.length];
91
92        for (int i = 0; i < grades.length; i++) {
93          boolean isDuplicate = false;
94          // Check if grade already exists in temp
95          for (int j = 0; j < uniqueCount; j++) {
96            if (grades[i] == temp[j]) {
97              isDuplicate = true;
98              break;
99            }
100         }
101         // If not duplicate, add it
102         if (!isDuplicate) {
103           temp[uniqueCount] = grades[i];
104           uniqueCount++;
105         }
106       }
107
108       // Create result array with correct size
109       int[] result = new int[uniqueCount];
110       for (int i = 0; i < uniqueCount; i++) {
111         result[i] = temp[i];
```

```
112      }
113
114      return result;
115    }
116  }
```

3. Task 3: Simple Bank Account Simulator

3.1. Preparation

Before implementing the solution, you must:

1. Design a class structure for the bank account system
2. Create a UML class diagram showing:
   - Class name (BankAccount)
   - All private attributes (balance, accountHolder, etc.)
   - All public methods with their parameters and return types
   - Constructor(s)
3. Transfer your implementation into this class structure
4. Only after completing the UML diagram should you begin coding

This preparation step ensures you think about encapsulation and class design before writing code.

3.2. Task

Create a simple bank account class that demonstrates encapsulation and method design. This task focuses on object-oriented programming principles.

Create a `BankAccount` class with the following features:

- Private balance variable
- Account holder name
- Methods for deposit, withdrawal, and balance inquiry
- Transaction validation (no negative deposits, sufficient funds for withdrawal)

Your program should:

1. Create a BankAccount object
2. Display a menu with options:
   - 1: Check balance
   - 2: Make deposit
   - 3: Make withdrawal
   - 4: Display account info
   - 0: Exit
3. Handle user input and call appropriate methods
4. Validate all transactions and display appropriate messages

3.3. Requirements

- Use private variables with public methods
- Validate all inputs (positive deposits, sufficient funds)
- Display clear success/error messages
- Use proper encapsulation principles

3.4. Assistance

Basic BankAccount class structure:

```java
public class BankAccount {
    private double balance;
    private String accountHolder;
```

```
4
5       public BankAccount(String name, double initialBalance) {
6           this.accountHolder = name;
7           this.balance = initialBalance;
8       }
9
10      public boolean deposit(double amount) {
11          if (amount > 0) {
12              balance += amount;
13              return true;
14          }
15          return false;
16      }
17
18      public boolean withdraw(double amount) {
19          if (amount > 0 && amount <= balance) {
20              balance -= amount;
21              return true;
22          }
23          return false;
24      }
25
26      public double getBalance() {
27          return balance;
28      }
29  }
```

3.5. Solution

BankAccount.java:

```java
1   public class BankAccount {
2     private double balance;
3     private String accountHolder;
4
5     // Constructor
6     public BankAccount(String accountHolder, double initialBalance) {
7       this.accountHolder = accountHolder;
8       if (initialBalance >= 0) {
9         this.balance = initialBalance;
10      } else {
11        this.balance = 0;
12        System.out.println("Initial balance cannot be negative. Set to 0.");
13      }
```

```
14    }
15
16    // Getter for balance
17    public double getBalance() {
18      return balance;
19    }
20
21    // Getter for account holder
22    public String getAccountHolder() {
23      return accountHolder;
24    }
25
26    // Deposit money
27    public boolean deposit(double amount) {
28      if (amount > 0) {
29        balance += amount;
30        System.out.println("Successfully deposited: $" + amount);
31        return true;
32      } else {
33        System.out.println("Deposit amount must be positive!");
34        return false;
35      }
36    }
37
38    // Withdraw money
39    public boolean withdraw(double amount) {
40      if (amount <= 0) {
41        System.out.println("Withdrawal amount must be positive!");
42        return false;
43      }
44
45      if (amount > balance) {
46        System.out.println("Insufficient funds! Current balance: $" + balance);
47        return false;
48      }
49
50      balance -= amount;
51      System.out.println("Successfully withdrew: $" + amount);
52      return true;
53    }
54
55    // Display account information
56    public void displayAccountInfo() {
```

```
57      System.out.println("\n=== Account Information ===");
58      System.out.println("Account Holder: " + accountHolder);
59      System.out.println("Current Balance: $" + balance);
60    }
61  }
```

BankAccountSimulator.java:

```java
import java.util.Scanner;

public class BankAccountSimulator {
  public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);

    // Create a bank account
    System.out.print("Enter account holder name: ");
    String name = scanner.nextLine();

    System.out.print("Enter initial balance: $");
    double initialBalance = scanner.nextDouble();

    BankAccount account = new BankAccount(name, initialBalance);

    boolean running = true;

    while (running) {
      System.out.println("\n=== Bank Account Menu ===");
      System.out.println("1: Check balance");
      System.out.println("2: Make deposit");
      System.out.println("3: Make withdrawal");
      System.out.println("4: Display account info");
      System.out.println("0: Exit");
      System.out.print("Your choice: ");

      int choice = scanner.nextInt();

      switch (choice) {
        case 0:
          System.out.println("Thank you for using our bank!");
          running = false;
          break;

        case 1:
          System.out.println("Current balance: $" + account.getBalance());
```

```
37            break;
38
39          case 2:
40            System.out.print("Enter deposit amount: $");
41            double depositAmount = scanner.nextDouble();
42            account.deposit(depositAmount);
43            break;
44
45          case 3:
46            System.out.print("Enter withdrawal amount: $");
47            double withdrawAmount = scanner.nextDouble();
48            account.withdraw(withdrawAmount);
49            break;
50
51          case 4:
52            account.displayAccountInfo();
53            break;
54
55          default:
56            System.out.println("Invalid choice!");
57        }
58      }
59
60      scanner.close();
61    }
62  }
```

4. Task 4: Array Sorting and Searching

Create a Java program that demonstrates array manipulation techniques including sorting and searching. This task will help you understand algorithm implementation and array processing.

- **Bubble sort**
  - Bubble Sort works by repeatedly stepping through the array, comparing adjacent elements and swapping them if they are in the wrong order, with larger elements "bubbling" to the end after each pass.
- **Linear Search**
  - Linear Search is a simple algorithm that examines each element in an array sequentially from the beginning until it finds the target value or reaches the end of the array, making it straightforward to implement but less efficient for large datasets.
- **Binary search**
  - Binary Search is an efficient algorithm that works on sorted arrays by repeatedly dividing the search space in half, comparing the target with the middle element to determine which half to search next. Both algorithms demonstrate different approaches to common programming problems with varying time complexities.

Write a program that:

- Creates an array of 10 random integers (1-100)
- Implements bubble sort to sort the array
- Implements linear search to find specific values
- Implements binary search (on sorted array)
- Displays arrays before/after sorting and search results

Your program should:

1. Generate and display an array of 10 random integers
2. Implement the following methods:
   - `void bubbleSort(int[] arr)` - sorts array using bubble sort
   - `int linearSearch(int[] arr, int target)` - returns index or −1
   - `int binarySearch(int[] arr, int target)` - returns index or −1 (requires sorted array)
   - `void printArray(int[] arr)` - displays array contents
3. Demonstrate all methods with user input for search values
4. Compare performance by counting comparisons in each search method

4.1. Requirements

- Use bubble sort algorithm for sorting
- Implement both linear and binary search
- Count and display number of comparisons for each search
- Handle cases where target value is not found

4.2. Assistance

Random array generation:

```java
import java.util.Random;

Random random = new Random();
int[] numbers = new int[10];
```

```
5  for (int i = 0; i < numbers.length; i++) {
6      numbers[i] = random.nextInt(100) + 1; // 1-100
7  }
```

4.3. Solution

Java

```
1   import java.util.Random;
2   import java.util.Scanner;
3
4   public class ArraySortingSearching {
5     public static void main(String[] args) {
6       Scanner scanner = new Scanner(System.in);
7       Random random = new Random();
8
9       // Generate random array
10      int[] numbers = new int[10];
11      System.out.println("=== Random Array Generation ===");
12      System.out.print("Original array: ");
13      for (int i = 0; i < numbers.length; i++) {
14        numbers[i] = random.nextInt(100) + 1;
15        System.out.print(numbers[i] + " ");
16      }
17      System.out.println();
18
19      // Bubble sort
20      System.out.println("\n=== Sorting with Bubble Sort ===");
21      bubbleSort(numbers);
22      System.out.print("Sorted array: ");
23      printArray(numbers);
24
25      // Search operations
26      boolean searching = true;
27      while (searching) {
28        System.out.println("\n=== Search Menu ===");
29        System.out.println("1: Linear Search");
30        System.out.println("2: Binary Search");
31        System.out.println("0: Exit");
32        System.out.print("Your choice: ");
33
34        int choice = scanner.nextInt();
35
36        if (choice == 0) {
37          searching = false;
38          continue;
```

```
39        }
40
41        System.out.print("Enter number to search for: ");
42        int target = scanner.nextInt();
43
44        if (choice == 1) {
45          // Linear search
46          int[] result = linearSearch(numbers, target);
47          int index = result[0];
48          int comparisons = result[1];
49
50          if (index != -1) {
51            System.out.println("Linear Search: Found at index " + index);
52          } else {
53            System.out.println("Linear Search: Not found");
54          }
55          System.out.println("Comparisons made: " + comparisons);
56
57        } else if (choice == 2) {
58          // Binary search
59          int[] result = binarySearch(numbers, target);
60          int index = result[0];
61          int comparisons = result[1];
62
63          if (index != -1) {
64            System.out.println("Binary Search: Found at index " + index);
65          } else {
66            System.out.println("Binary Search: Not found");
67          }
68          System.out.println("Comparisons made: " + comparisons);
69        }
70      }
71
72      scanner.close();
73    }
74
75    // Bubble sort algorithm
76    public static void bubbleSort(int[] arr) {
77      int n = arr.length;
78      for (int i = 0; i < n - 1; i++) {
79        for (int j = 0; j < n - i - 1; j++) {
80          if (arr[j] > arr[j + 1]) {
81            // Swap elements
```

```
82              int temp = arr[j];
83              arr[j] = arr[j + 1];
84              arr[j + 1] = temp;
85            }
86          }
87        }
88      }
89
90      // Linear search - returns [index, comparisons]
91      public static int[] linearSearch(int[] arr, int target) {
92        int comparisons = 0;
93        for (int i = 0; i < arr.length; i++) {
94          comparisons++;
95          if (arr[i] == target) {
96            return new int[]{i, comparisons};
97          }
98        }
99        return new int[]{-1, comparisons};
100     }
101
102     // Binary search - returns [index, comparisons]
103     public static int[] binarySearch(int[] arr, int target) {
104       int left = 0;
105       int right = arr.length - 1;
106       int comparisons = 0;
107
108       while (left <= right) {
109         comparisons++;
110         int mid = left + (right - left) / 2;
111
112         if (arr[mid] == target) {
113           return new int[]{mid, comparisons};
114         }
115
116         if (arr[mid] < target) {
117           left = mid + 1;
118         } else {
119           right = mid - 1;
120         }
121       }
122
123       return new int[]{-1, comparisons};
124     }
```

```
125
126   // Print array
127   public static void printArray(int[] arr) {
128     for (int num : arr) {
129       System.out.print(num + " ");
130     }
131     System.out.println();
132   }
133 }
```

5. Task 5: Object Comparison and Equality

Develop a comprehensive understanding of object comparison in Java by creating a Student class that implements proper equality checking and comparison methods.

Create a `Student` class with the following features:

- Private fields: studentID (int), name (String), grade (double)
- Override `equals()` and `hashCode()` methods
- Create methods to compare students by different criteria

Your program should:

1. Create an array of Student objects with sample data
2. Demonstrate equality checking between students
3. Sort students by grade (natural ordering)
4. Sort students by name using a custom comparator
5. Find duplicate students and display them
6. Search for students by ID and name

5.1. Requirements

- Override `equals()` correctly
- Create custom comparators for different sorting criteria
- Demonstrate all comparison methods with test data

5.2. Assistance

Example Student class with equals:

```java
public class Student {
    private int studentID;
    private String name;
    private double grade;

    public Student(int studentID, String name, double grade) {
        this.studentID = studentID;
        this.name = name;
        this.grade = grade;
    }

}
```

5.3. Solution

Student.java:

```java
public class Student {
  private int studentID;
  private String name;
  private double grade;

```

```
6     // Constructor
7     public Student(int studentID, String name, double grade) {
8       this.studentID = studentID;
9       this.name = name;
10      this.grade = grade;
11    }
12
13    // Getters
14    public int getStudentID() {
15      return studentID;
16    }
17
18    public String getName() {
19      return name;
20    }
21
22    public double getGrade() {
23      return grade;
24    }
25
26    // Override equals method - students are equal if they have the same ID
27    @Override
28    public boolean equals(Object obj) {
29      if (this == obj) return true;
30      if (obj == null || getClass() != obj.getClass()) return false;
31
32      Student student = (Student) obj;
33      return studentID == student.studentID;
34    }
35
36    // Override hashCode method
37    @Override
38    public int hashCode() {
39      return studentID;
40    }
41
42    // Display student info
43    public void displayInfo() {
44      System.out.println("ID: " + studentID + ", Name: " + name + ", Grade: " +
  grade);
45    }
46  }
```

StudentComparison.java:

```java
public class StudentComparison {
  public static void main(String[] args) {
    // Create array of students
    Student[] students = new Student[5];
    students[0] = new Student(101, "Alice", 85.5);
    students[1] = new Student(102, "Bob", 92.0);
    students[2] = new Student(103, "Charlie", 78.5);
    students[3] = new Student(101, "Alice Duplicate", 90.0); // Same ID as student 0
    students[4] = new Student(104, "Diana", 88.0);

    System.out.println("=== Original Student List ===");
    displayAllStudents(students);

    // Test equality
    System.out.println("\n=== Testing Equality ===");
    System.out.println("Student 0 equals Student 3? " + students[0].equals(students[3]));
    System.out.println("Student 0 equals Student 1? " + students[0].equals(students[1]));

    // Find duplicates
    System.out.println("\n=== Finding Duplicates ===");
    findDuplicates(students);

    // Sort by grade
    System.out.println("\n=== Sorted by Grade (Ascending) ===");
    sortByGrade(students);
    displayAllStudents(students);

    // Sort by name
    System.out.println("\n=== Sorted by Name (Alphabetically) ===");
    sortByName(students);
    displayAllStudents(students);

    // Search by ID
    System.out.println("\n=== Search by ID ===");
    Student found = searchByID(students, 103);
    if (found != null) {
      System.out.print("Found: ");
      found.displayInfo();
    } else {
      System.out.println("Not found");
```

```
41      }
42
43      // Search by name
44      System.out.println("\n=== Search by Name ===");
45      found = searchByName(students, "Bob");
46      if (found != null) {
47        System.out.print("Found: ");
48        found.displayInfo();
49      } else {
50        System.out.println("Not found");
51      }
52    }
53
54    // Display all students
55    public static void displayAllStudents(Student[] students) {
56      for (Student s : students) {
57        s.displayInfo();
58      }
59    }
60
61    // Find and display duplicate students
62    public static void findDuplicates(Student[] students) {
63      for (int i = 0; i < students.length; i++) {
64        for (int j = i + 1; j < students.length; j++) {
65          if (students[i].equals(students[j])) {
66            System.out.println("Duplicate found:");
67            System.out.print("  ");
68            students[i].displayInfo();
69            System.out.print("  ");
70            students[j].displayInfo();
71          }
72        }
73      }
74    }
75
76    // Sort students by grade (bubble sort)
77    public static void sortByGrade(Student[] students) {
78      for (int i = 0; i < students.length - 1; i++) {
79        for (int j = 0; j < students.length - i - 1; j++) {
80          if (students[j].getGrade() > students[j + 1].getGrade()) {
81            Student temp = students[j];
82            students[j] = students[j + 1];
83            students[j + 1] = temp;
```

```
84            }
85          }
86        }
87      }
88
89      // Sort students by name (bubble sort)
90      public static void sortByName(Student[] students) {
91        for (int i = 0; i < students.length - 1; i++) {
92          for (int j = 0; j < students.length - i - 1; j++) {
93            if (students[j].getName().compareTo(students[j + 1].getName()) > 0) {
94              Student temp = students[j];
95              students[j] = students[j + 1];
96              students[j + 1] = temp;
97            }
98          }
99        }
100     }
101
102     // Search for student by ID
103     public static Student searchByID(Student[] students, int id) {
104       for (Student s : students) {
105         if (s.getStudentID() == id) {
106           return s;
107         }
108       }
109       return null;
110     }
111
112     // Search for student by name
113     public static Student searchByName(Student[] students, String name) {
114       for (Student s : students) {
115         if (s.getName().equals(name)) {
116           return s;
117         }
118       }
119       return null;
120     }
121 }
```

6. Task 6: Simple Inventory Management System

6.1. Preparation

Before implementing the solution, you must:

1. Design the complete class structure for the inventory management system
2. Create UML class diagrams for both classes showing:
   - **Item class**:
     - All private attributes (name, itemID, quantity, price, static variables)
     - All public methods with their parameters and return types
     - Constructor(s)
   - **Inventory class**:
     - All private attributes (items array, itemCount, etc.)
     - All public methods with their parameters and return types
     - Private helper methods (e.g., resizeArray)
     - Constructor(s)
3. Show the relationship between the two classes (composition/aggregation)
4. Transfer your implementation into this class structure
5. Only after completing the UML diagrams should you begin coding

This preparation step ensures you think about multi-class system design and relationships before writing code.

6.2. Task

Create a comprehensive inventory management system that combines all concepts learned in previous tasks. This capstone task integrates arrays, methods, encapsulation, and object-oriented design.

Develop an `Item` class and `Inventory` class to manage a collection of items with the following features:

- Item class with name, ID, quantity, and price
- Inventory class to manage multiple items
- Methods for adding, removing, updating, and searching items
- Generate reports and handle low stock warnings

Your program should:

1. Create an Inventory system with Item objects
2. Implement the following functionality:
   - Add new items to inventory
   - Update item quantities (restock/sell)
   - Search items by name or ID
   - Display low stock warnings (quantity < 5)
   - Calculate total inventory value
   - Generate inventory reports
3. Use arrays to store items and implement dynamic resizing
4. Provide a menu-driven interface for all operations

6.3. Requirements

- Use proper encapsulation with private variables
- Implement input validation for all operations
- Handle array resizing when adding new items

- Calculate and display inventory statistics
- Use static variables to track total number of items

6.4. Assistance

Item class structure:

```java
public class Item {
    private String name;
    private int itemID;
    private int quantity;
    private double price;
    private static int totalItems = 0;

    public Item(String name, int itemID, int quantity, double price) {
        this.name = name;
        this.itemID = itemID;
        this.quantity = quantity;
        this.price = price;
        totalItems++;
    }

    public double getTotalValue() {
        return quantity * price;
    }

    public boolean isLowStock() {
        return quantity < 5;
    }

    // Getters and setters...
}
```

Inventory class with dynamic array:

```java
public class Inventory {
    private Item[] items;
    private int itemCount;
    private static final int INITIAL_CAPACITY = 10;

    public Inventory() {
        items = new Item[INITIAL_CAPACITY];
        itemCount = 0;
    }

    public void addItem(Item item) {
```

```
12          if (itemCount >= items.length) {
13              resizeArray();
14          }
15          items[itemCount++] = item;
16      }
17
18      private void resizeArray() {
19          Item[] newItems = new Item[items.length * 2];
20          System.arraycopy(items, 0, newItems, 0, itemCount);
21          items = newItems;
22      }
23
24      public Item findItemByID(int id) {
25          for (int i = 0; i < itemCount; i++) {
26              if (items[i].getItemID() == id) {
27                  return items[i];
28              }
29          }
30          return null;
31      }
32  }
```

6.5. Solution

Item.java:

```
1   public class Item {                                                    Java
2     private String name;
3     private int itemID;
4     private int quantity;
5     private double price;
6     private static int totalItems = 0;
7
8     // Constructor
9     public Item(String name, int itemID, int quantity, double price) {
10      this.name = name;
11      this.itemID = itemID;
12      this.quantity = quantity;
13      this.price = price;
14      totalItems++;
15    }
16
17    // Getters
18    public String getName() {
```

```
19      return name;
20    }
21
22    public int getItemID() {
23      return itemID;
24    }
25
26    public int getQuantity() {
27      return quantity;
28    }
29
30    public double getPrice() {
31      return price;
32    }
33
34    public static int getTotalItems() {
35      return totalItems;
36    }
37
38    // Setters
39    public void setName(String name) {
40      this.name = name;
41    }
42
43    public void setQuantity(int quantity) {
44      if (quantity >= 0) {
45        this.quantity = quantity;
46      } else {
47        System.out.println("Quantity cannot be negative!");
48      }
49    }
50
51    public void setPrice(double price) {
52      if (price >= 0) {
53        this.price = price;
54      } else {
55        System.out.println("Price cannot be negative!");
56      }
57    }
58
59    // Calculate total value of this item
60    public double getTotalValue() {
61      return quantity * price;
```

```
62    }
63
64    // Check if item is low stock
65    public boolean isLowStock() {
66      return quantity < 5;
67    }
68
69    // Add stock
70    public void restock(int amount) {
71      if (amount > 0) {
72        quantity += amount;
73        System.out.println("Restocked " + amount + " units of " + name);
74      }
75    }
76
77    // Sell item (reduce stock)
78    public boolean sell(int amount) {
79      if (amount <= 0) {
80        System.out.println("Amount must be positive!");
81        return false;
82      }
83      if (amount > quantity) {
84        System.out.println("Insufficient stock! Available: " + quantity);
85        return false;
86      }
87      quantity -= amount;
88      System.out.println("Sold " + amount + " units of " + name);
89      return true;
90    }
91
92    // Display item info
93    public void displayInfo() {
94      System.out.printf("ID: %d | Name: %-15s | Qty: %3d | Price: $%.2f | Value: $
    %.2f%s\n",
95          itemID, name, quantity, price, getTotalValue(),
96          isLowStock() ? " [LOW STOCK]" : "");
97    }
98  }
```

Inventory.java:

```
1   public class Inventory {                                          Java
2     private Item[] items;
3     private int itemCount;
```

```
4     private static final int INITIAL_CAPACITY = 10;
5
6     // Constructor
7     public Inventory() {
8       items = new Item[INITIAL_CAPACITY];
9       itemCount = 0;
10    }
11
12    // Add item to inventory
13    public void addItem(Item item) {
14      // Check if item already exists
15      Item existing = findItemByID(item.getItemID());
16      if (existing != null) {
17        System.out.println("Item with ID " + item.getItemID() + " already
    exists!");
18        return;
19      }
20
21      // Resize if necessary
22      if (itemCount >= items.length) {
23        resizeArray();
24      }
25
26      items[itemCount++] = item;
27      System.out.println("Item added successfully!");
28    }
29
30    // Resize array when full
31    private void resizeArray() {
32      Item[] newItems = new Item[items.length * 2];
33      System.arraycopy(items, 0, newItems, 0, itemCount);
34      items = newItems;
35      System.out.println("Inventory capacity expanded to " + items.length);
36    }
37
38    // Find item by ID
39    public Item findItemByID(int id) {
40      for (int i = 0; i < itemCount; i++) {
41        if (items[i].getItemID() == id) {
42          return items[i];
43        }
44      }
45      return null;
```

```
46    }
47
48    // Find item by name
49    public Item findItemByName(String name) {
50      for (int i = 0; i < itemCount; i++) {
51        if (items[i].getName().equalsIgnoreCase(name)) {
52          return items[i];
53        }
54      }
55      return null;
56    }
57
58    // Remove item by ID
59    public boolean removeItem(int id) {
60      for (int i = 0; i < itemCount; i++) {
61        if (items[i].getItemID() == id) {
62          // Shift elements left
63          for (int j = i; j < itemCount - 1; j++) {
64            items[j] = items[j + 1];
65          }
66          items[--itemCount] = null;
67          System.out.println("Item removed successfully!");
68          return true;
69        }
70      }
71      System.out.println("Item not found!");
72      return false;
73    }
74
75    // Display all items
76    public void displayInventory() {
77      if (itemCount == 0) {
78        System.out.println("Inventory is empty!");
79        return;
80      }
81
82      System.out.println("\n=== Current Inventory ===");
83      for (int i = 0; i < itemCount; i++) {
84        items[i].displayInfo();
85      }
86    }
87
88    // Display low stock warnings
```

```java
    public void displayLowStockWarnings() {
      System.out.println("\n=== Low Stock Warnings ===");
      boolean foundLowStock = false;

      for (int i = 0; i < itemCount; i++) {
        if (items[i].isLowStock()) {
          items[i].displayInfo();
          foundLowStock = true;
        }
      }

      if (!foundLowStock) {
        System.out.println("No items with low stock!");
      }
    }

    // Calculate total inventory value
    public double calculateTotalValue() {
      double total = 0;
      for (int i = 0; i < itemCount; i++) {
        total += items[i].getTotalValue();
      }
      return total;
    }

    // Generate inventory report
    public void generateReport() {
      System.out.println("\n=== Inventory Report ===");
      System.out.println("Total Items: " + itemCount);
      System.out.println("Total Inventory Value: $" + calculateTotalValue());
      System.out.println("Inventory Capacity: " + items.length);

      displayInventory();
      displayLowStockWarnings();
    }

    public int getItemCount() {
      return itemCount;
    }
}
```

InventoryManagementSystem.java:

```java
import java.util.Scanner;
```

```
2
3   public class InventoryManagementSystem {
4     public static void main(String[] args) {
5       Scanner scanner = new Scanner(System.in);
6       Inventory inventory = new Inventory();
7
8       // Add some sample items
9       inventory.addItem(new Item("Laptop", 1001, 15, 899.99));
10      inventory.addItem(new Item("Mouse", 1002, 50, 19.99));
11      inventory.addItem(new Item("Keyboard", 1003, 3, 49.99));
12      inventory.addItem(new Item("Monitor", 1004, 8, 299.99));
13
14      boolean running = true;
15
16      while (running) {
17        System.out.println("\n=== Inventory Management System ===");
18        System.out.println("1: Add new item");
19        System.out.println("2: Update item quantity (restock)");
20        System.out.println("3: Sell item");
21        System.out.println("4: Search item by ID");
22        System.out.println("5: Search item by name");
23        System.out.println("6: Display all items");
24        System.out.println("7: Display low stock warnings");
25        System.out.println("8: Generate inventory report");
26        System.out.println("9: Remove item");
27        System.out.println("0: Exit");
28        System.out.print("Your choice: ");
29
30        int choice = scanner.nextInt();
31        scanner.nextLine(); // Consume newline
32
33        switch (choice) {
34          case 0:
35            System.out.println("Exiting system...");
36            running = false;
37            break;
38
39          case 1:
40            // Add new item
41            System.out.print("Enter item name: ");
42            String name = scanner.nextLine();
43            System.out.print("Enter item ID: ");
44            int id = scanner.nextInt();
```

```
45            System.out.print("Enter quantity: ");
46            int quantity = scanner.nextInt();
47            System.out.print("Enter price: $");
48            double price = scanner.nextDouble();
49            scanner.nextLine();
50
51            inventory.addItem(new Item(name, id, quantity, price));
52            break;
53
54          case 2:
55            // Restock item
56            System.out.print("Enter item ID to restock: ");
57            int restockID = scanner.nextInt();
58            Item restockItem = inventory.findItemByID(restockID);
59            if (restockItem != null) {
60              System.out.print("Enter quantity to add: ");
61              int restockAmount = scanner.nextInt();
62              restockItem.restock(restockAmount);
63            } else {
64              System.out.println("Item not found!");
65            }
66            break;
67
68          case 3:
69            // Sell item
70            System.out.print("Enter item ID to sell: ");
71            int sellID = scanner.nextInt();
72            Item sellItem = inventory.findItemByID(sellID);
73            if (sellItem != null) {
74              System.out.print("Enter quantity to sell: ");
75              int sellAmount = scanner.nextInt();
76              sellItem.sell(sellAmount);
77            } else {
78              System.out.println("Item not found!");
79            }
80            break;
81
82          case 4:
83            // Search by ID
84            System.out.print("Enter item ID: ");
85            int searchID = scanner.nextInt();
86            Item foundByID = inventory.findItemByID(searchID);
87            if (foundByID != null) {
```

```
88              System.out.println("\nItem found:");
89              foundByID.displayInfo();
90            } else {
91              System.out.println("Item not found!");
92            }
93            break;
94
95          case 5:
96            // Search by name
97            System.out.print("Enter item name: ");
98            String searchName = scanner.nextLine();
99            Item foundByName = inventory.findItemByName(searchName);
100           if (foundByName != null) {
101             System.out.println("\nItem found:");
102             foundByName.displayInfo();
103           } else {
104             System.out.println("Item not found!");
105           }
106           break;
107
108         case 6:
109           // Display all items
110           inventory.displayInventory();
111           break;
112
113         case 7:
114           // Display low stock warnings
115           inventory.displayLowStockWarnings();
116           break;
117
118         case 8:
119           // Generate report
120           inventory.generateReport();
121           break;
122
123         case 9:
124           // Remove item
125           System.out.print("Enter item ID to remove: ");
126           int removeID = scanner.nextInt();
127           inventory.removeItem(removeID);
128           break;
129
130         default:
```

```
131             System.out.println("Invalid choice!");
132         }
133     }
134
135     scanner.close();
136   }
137 }
```

7. Lab Execution

If your program is not yet working without issue, we will try to correct this during the course of the lab. With good preparation, this should not be a problem. Every student is required to be able to explain their thought process at the beginning of the lab. By the end of the lab, the task needs to be completed. Of course, we will support you, but your personal commitment must also be clearly recognizable! Julian Moldenhauer, Furkan Yildirim, and Emily Antosch wish you lots of fun and success!